

Can KAN Handle the Curve?

Gonzalo Alvis

Facultad de Ingeniería
Universidad del Pacífico
Lima, Perú

gr.alvisbustamante@alum.up.edu.pe

Franz Figueroa

Facultad de Ingeniería
Universidad del Pacífico
Lima, Perú

f.figueroag@alum.up.edu.pe

David Huamán

Facultad de Ingeniería
Universidad del Pacífico
Lima, Perú

d.huamanor@alum.up.edu.pe

Fabrizio Marin

Facultad de Ingeniería
Universidad del Pacífico
Lima, Perú

sf.marinc@alum.up.edu.pe

Abstract—El entrenamiento de modelos de aprendizaje profundo es fundamentalmente un problema de optimización numérica. Este trabajo presenta un estudio comparativo entre la arquitectura emergente Kolmogorov-Arnold Networks (KAN) y los tradicionales Perceptrones Multicapa (MLP), evaluando su desempeño bajo distintos paradigmas de optimización: métodos de primer orden (SGD, Adam, AdamW) y métodos de segundo orden (L-BFGS). Mediante una metodología de dos fases, se analizó primero la capacidad de aproximación funcional y convergencia en superficies sintéticas complejas (incluyendo el valle de Rosenbrock), y posteriormente la escalabilidad en la clasificación de imágenes (MNIST). Los resultados experimentales demuestran que, si bien los métodos estocásticos son eficientes para tareas a gran escala, el optimizador L-BFGS ofrece una superioridad crítica en precisión y estabilidad para topologías mal condicionadas, permitiendo a las KAN recuperar estructuras simbólicas interpretables que los MLPs no pueden capturar. Se concluye que, aunque KAN presenta un mayor costo computacional, su combinación con optimizadores de segundo orden ofrece una alternativa robusta para problemas que requieren alta precisión y explicabilidad científica.

Index Terms—Kolmogorov-Arnold Networks, Multi-layer Perceptron, Optimización Numérica, L-BFGS, Interpretabilidad, Deep Learning.

I. INTRODUCCIÓN

El entrenamiento de modelos de machine learning constituye esencialmente un problema de optimización numérica. En particular, la mayoría de métodos de aprendizaje supervisado buscan minimizar una función de pérdida, de alta dimensionalidad y con una estructura geométrica compleja. Esto ha llevado a que el estudio de algoritmos de optimización sea un componente central dentro del desarrollo de modelos eficientes, estables y capaces de generalizar. En este contexto, el rendimiento de un modelo no depende únicamente de su arquitectura, sino también —y en muchos casos de forma determinante— del algoritmo de optimización utilizado durante el proceso de entrenamiento.

Históricamente, los métodos de primera orden, como *Stochastic Gradient Descent* (SGD) [1] y sus variantes, han sido ampliamente utilizados en redes neuronales debido a su escalabilidad, simplicidad y eficiencia computacional. La introducción de optimizadores adaptativos como Adam [2] representó un avance significativo al permitir una convergencia más rápida en problemas con gradientes ruidosos o mal condicionados. Posteriormente, enfoques refinados como AdamW [3] corrigieron deficiencias asociadas al manejo del decaimiento de pesos,

contribuyendo a mejorar la capacidad de generalización de los modelos.

Sin embargo, el dominio de los optimizadores de primera orden no ha eliminado el interés por los métodos de segunda orden, como BFGS [4] y L-BFGS [5], clásicos en la optimización matemática. Estos métodos se basan en aproximaciones de la matriz Hessiana para capturar la curvatura local de la función de pérdida, lo que puede conducir a trayectorias de optimización más estables y a una convergencia más precisa en etapas finales. A pesar de su mayor costo computacional, los avances modernos en hardware, las implementaciones eficientes y la tendencia hacia modelos más compactos han motivado que estas técnicas resurjan como alternativas relevantes en escenarios donde la precisión y estabilidad son críticas.

Al mismo tiempo, la arquitectura de los modelos ha evolucionado. Los *Multilayer Perceptrons* (MLP) [6] continúan siendo una referencia fundamental, tanto por su capacidad universal de aproximación como por su utilidad para analizar el comportamiento de diferentes optimizadores en un entorno controlado. No obstante, nuevas arquitecturas, como los Kolmogorov-Arnold Networks (KAN) [7], han propuesto un enfoque radicalmente distinto, sustituyendo las transformaciones lineales por representaciones funcionales unidimensionales basadas en principios teóricos de la descomposición de funciones. Esta estructura genera una geometría de la superficie de pérdida potencialmente diferente, lo que abre preguntas sobre qué optimizadores se adaptan mejor a este tipo de modelos. En este escenario, resulta pertinente realizar una comparación sistemática y rigurosa entre distintos algoritmos de optimización aplicados a arquitecturas contrastantes. Evaluar el desempeño de optimizadores de primera y segunda orden permite no solo determinar cuál ofrece mejores resultados en términos de velocidad de convergencia, estabilidad y capacidad de generalización, sino también entender cómo interactúan las características del optimizador con la estructura interna del modelo.

El presente proyecto tiene como objetivo analizar el comportamiento de cuatro estrategias de optimización: SGD, Adam, AdamW y L-BFGS, aplicadas al entrenamiento de MLP y KAN en una tarea supervisada. Mediante la evaluación de métricas cuantitativas y el análisis de la trayectoria de entrenamiento, se espera identificar patrones, ventajas y limitaciones de cada método, así como aportar evidencia experimen-

tal que contribuya a una mejor selección de optimizadores según el tipo de modelo y las características del problema. Esta investigación se enmarca en la creciente necesidad de comprender la interacción entre algoritmos de optimización y arquitecturas emergentes de redes neuronales, un aspecto fundamental para avanzar hacia modelos más eficientes, interpretables y robustos en aplicaciones reales.

II. ESTADO DEL ARTE

Varios estudios recientes han profundizado en la comparación empírica entre algoritmos de optimización, evaluando su rendimiento en tareas supervisadas bajo diferentes arquitecturas de redes neuronales. Si bien optimizadores como SGD, Adam y AdamW son ampliamente utilizados por su eficiencia y simplicidad computacional, investigaciones recientes han puesto de manifiesto que su desempeño puede verse superado en escenarios específicos por métodos de segundo orden, como BFGS y L-BFGS, especialmente en fases finales del entrenamiento o en problemas mal condicionados. Por ejemplo, Kiyani et al. [8] demostraron que, en el contexto de redes neuronales informadas por física (PINNs), el uso de BFGS permitió reducir la pérdida en varios órdenes de magnitud frente a Adam, evidenciando una convergencia más precisa. En esta misma línea, se ha observado que L-BFGS logra soluciones más estables cuando se entrena con batches suficientemente grandes, a pesar de su mayor costo computacional.

Además, son relevantes los estudios enfocados en arquitecturas emergentes como las Kolmogorov–Arnold Networks (KANs), donde la geometría de la superficie de pérdida difiere significativamente de la de los MLP tradicionales. Liu et al. [7], en su implementación original de KANs, adoptaron L-BFGS como optimizador principal, y estudios posteriores como el de Noorizadegan et al. [9] recomiendan esquemas escalonados Adam+L-BFGS como configuración óptima para este tipo de redes funcionales. Estas estrategias también han demostrado ser eficaces en la solución de ecuaciones diferenciales parciales y otros problemas con estructuras complejas. En conjunto, estos trabajos reflejan una tendencia clara hacia la adopción de enfoques mixtos de optimización, ajustados tanto al tipo de tarea como a la arquitectura del modelo. Nuestro proyecto se alinea con esta dirección al analizar el comportamiento de algoritmos aplicados a MLP y KAN, con el objetivo de identificar patrones de convergencia, estabilidad y generalización que guíen una elección más informada del optimizador según el contexto.

III. MARCO TEÓRICO

A. Redes Neuronales

Las redes neuronales artificiales son modelos computacionales inspirados en el funcionamiento del cerebro humano, capaces de aprender representaciones complejas de datos a través de transformaciones no lineales sucesivas [10]. Estos modelos han demostrado ser altamente efectivos en una amplia variedad de tareas, desde visión por computadora hasta procesamiento de lenguaje natural.

1) *Perceptrón Multicapa (MLP)*: El Perceptrón Multicapa es la arquitectura fundamental de redes neuronales feedforward, compuesta por múltiples capas de neuronas conectadas secuencialmente [11]. Un MLP con L capas ocultas se puede expresar matemáticamente como:

$$f(\mathbf{x}) = \sigma_L(W_L \sigma_{L-1}(W_{L-1} \cdots \sigma_1(W_1 \mathbf{x} + b_1) \cdots + b_{L-1}) + b_L) \quad (1)$$

donde $\mathbf{x}$ es el vector de entrada, W_i y b_i son las matrices de pesos y vectores de sesgo de la capa i , y σ_i son funciones de activación no lineales. Las funciones de activación más comunes incluyen ReLU, sigmoide y tangente hiperbólica [12].

La operación fundamental en cada neurona del MLP es una transformación afín seguida de una no linealidad:

$$h_i = \sigma(w_i^T x + b_i) \quad (2)$$

Esta arquitectura ha sido ampliamente estudiada y se ha demostrado que los MLP son aproximadores universales de funciones, es decir, pueden aproximar cualquier función continua con precisión arbitraria dado un número suficiente de neuronas [13]. Sin embargo, los MLP tradicionales enfrentan limitaciones en cuanto a interpretabilidad y pueden requerir arquitecturas muy profundas para capturar relaciones complejas en los datos.

2) *Kolmogorov-Arnold Networks (KAN)*: Las Kolmogorov-Arnold Networks representan un paradigma alternativo inspirado en el teorema de representación de Kolmogorov-Arnold, el cual establece que cualquier función multivariada continua puede expresarse como una superposición finita de funciones continuas de una sola variable [7], [14].

Matemáticamente, el teorema de Kolmogorov-Arnold establece que cualquier función continua $f : [0, 1]^n \rightarrow \mathbb{R}$ puede representarse como:

$$f(x_1, \dots, x_n) = \sum_{q=0}^{2n} \Phi_q \left(\sum_{p=1}^n \phi_{q,p}(x_p) \right) \quad (3)$$

donde $\phi_{q,p} : [0, 1] \rightarrow \mathbb{R}$ y $\Phi_q : \mathbb{R} \rightarrow \mathbb{R}$ son funciones univariadas continuas.

A diferencia de los MLP, donde las funciones de activación están fijas y los pesos son aprendibles, en las KAN las funciones de activación en sí mismas son aprendibles [7]. Cada borde de la red contiene una función univariada parametrizada, comúnmente representada mediante B-splines:

$$\Phi(\mathbf{x}) = \mathbf{W} \cdot \sigma(\mathbf{x}) + \mathbf{b} \cdot \text{spline}(\mathbf{x}) \quad (4)$$

donde $\text{spline}(\mathbf{x})$ es una combinación lineal de funciones base B-spline. Esta representación permite que las KAN aprendan funciones de activación adaptativas específicas para cada conexión de la red.

Las KAN ofrecen ventajas potenciales en términos de interpretabilidad, ya que cada función aprendible puede visualizarse y analizarse individualmente, permitiendo comprender mejor cómo la red procesa la información [7]. Además, teóricamente pueden lograr mayor precisión con

menos parámetros comparadas con MLP en ciertos tipos de problemas, especialmente aquellos con estructura subyacente suave y composicional.

B. El Problema de Optimización

El entrenamiento de ambas arquitecturas, MLP y KAN, requiere resolver un problema de optimización no convexa de la forma:

$$\theta^* = \arg \min_{\theta} \mathcal{L}(\theta) = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N \ell(f_{\theta}(x_i), y_i) \quad (5)$$

donde θ representa todos los parámetros aprendibles del modelo (pesos, sesgos, y en el caso de KAN, también los parámetros de las funciones spline), ℓ es una función de pérdida que mide la discrepancia entre las predicciones $f_{\theta}(x_i)$ y las etiquetas verdaderas y_i , y N es el número de muestras de entrenamiento.

La naturaleza no convexa del espacio de parámetros, especialmente en redes profundas, implica la existencia de múltiples mínimos locales y regiones planas (plateaus), lo que hace que la elección del algoritmo de optimización sea crítica para el éxito del entrenamiento [10]. Además, la diferente estructura paramétrica entre MLP y KAN puede resultar en paisajes de optimización con características distintas, lo que motiva la comparación de diferentes métodos de optimización para ambas arquitecturas.

En este sentido, el entrenamiento de redes neuronales es esencialmente un problema de optimización no convexa en alta dimensión, donde se busca minimizar una función de pérdida $\mathcal{L}(\theta)$ con respecto a los parámetros del modelo θ [10]. La elección del algoritmo de optimización es crucial para el rendimiento y la eficiencia del entrenamiento, ya que determina cómo se actualizan los parámetros en cada iteración.

C. Métodos de Primer Orden

Los métodos de primer orden utilizan únicamente información del gradiente y son especialmente adecuados para problemas de gran escala debido a su bajo costo computacional por iteración [15].

1) *SGD (Stochastic Gradient Descent)*: El Descenso de Gradiente Estocástico es el método fundamental de optimización en aprendizaje profundo [16]. En cada iteración, SGD actualiza los parámetros utilizando el gradiente calculado sobre un mini-batch de datos:

$$\theta_{k+1} = \theta_k - \eta \nabla \mathcal{L}(\theta_k; \mathcal{B}_k) \quad (6)$$

donde η es la tasa de aprendizaje y $\mathcal{B}_k$ es un mini-batch muestreado aleatoriamente. Una variante común incluye momentum [17]:

$$\begin{aligned} v_{k+1} &= \beta v_k + \nabla \mathcal{L}(\theta_k; \mathcal{B}_k) \\ \theta_{k+1} &= \theta_k - \eta v_{k+1} \end{aligned} \quad (7)$$

donde β (típicamente 0.9) controla la contribución del gradiente histórico. El momentum acelera el aprendizaje en direcciones consistentes y amortigua las oscilaciones [18].

2) *Adam (Adaptive Moment Estimation)*: Adam combina las ventajas de dos extensiones de SGD: AdaGrad, que adapta las tasas de aprendizaje basándose en la frecuencia de las actualizaciones de parámetros, y RMSProp, que mantiene una media móvil de gradientes al cuadrado [2]. El algoritmo mantiene estimaciones del primer momento (media) y segundo momento (varianza no centrada) de los gradientes:

$$\begin{aligned} m_k &= \beta_1 m_{k-1} + (1 - \beta_1) \nabla \mathcal{L}(\theta_k) \\ v_k &= \beta_2 v_{k-1} + (1 - \beta_2) (\nabla \mathcal{L}(\theta_k))^2 \end{aligned} \quad (8)$$

Para corregir el sesgo de inicialización hacia cero, se aplican correcciones de sesgo:

$$\hat{m}_k = \frac{m_k}{1 - \beta_1^k}, \quad \hat{v}_k = \frac{v_k}{1 - \beta_2^k} \quad (9)$$

La actualización de parámetros se realiza mediante:

$$\theta_{k+1} = \theta_k - \frac{\eta}{\sqrt{\hat{v}_k} + \epsilon} \hat{m}_k \quad (10)$$

donde los valores típicos son $\beta_1 = 0.9$, $\beta_2 = 0.999$, y $\epsilon = 10^{-8}$. Adam ha demostrado ser robusto en una amplia variedad de problemas y es el optimizador por defecto en muchas aplicaciones de aprendizaje profundo [15].

D. Métodos de Segundo Orden

Los métodos de segundo orden utilizan información de la matriz Hessiana para guiar la búsqueda del mínimo, proporcionando una aproximación cuadrática de la función objetivo [19].

1) *BFGS (Broyden-Fletcher-Goldfarb-Shanno)*: El algoritmo BFGS es un método quasi-Newton que aproxima la matriz Hessiana inversa mediante actualizaciones sucesivas, evitando el costo computacional de calcular la Hessiana directamente [19]. La actualización de parámetros en BFGS sigue la regla:

$$\theta_{k+1} = \theta_k - \alpha_k H_k^{-1} \nabla \mathcal{L}(\theta_k) \quad (11)$$

donde H_k^{-1} es la aproximación de la Hessiana inversa en la iteración k , y α_k es el tamaño de paso determinado mediante búsqueda de línea. La matriz H_k^{-1} se actualiza mediante la fórmula de BFGS:

$$H_{k+1}^{-1} = \left(I - \frac{s_k y_k^T}{y_k^T s_k} \right) H_k^{-1} \left(I - \frac{y_k s_k^T}{y_k^T s_k} \right) + \frac{s_k s_k^T}{y_k^T s_k} \quad (12)$$

donde $s_k = \theta_{k+1} - \theta_k$ y $y_k = \nabla \mathcal{L}(\theta_{k+1}) - \nabla \mathcal{L}(\theta_k)$.

Una limitación importante de BFGS es su complejidad de memoria $O(n^2)$, donde n es el número de parámetros, lo que lo hace impracticable para redes neuronales con millones de parámetros [20].

2) *L-BFGS (Limited-memory BFGS)*: L-BFGS es una variante de memoria limitada del algoritmo BFGS diseñada específicamente para problemas de optimización en alta dimensión [20]. En lugar de almacenar la matriz Hessiana inversa completa, L-BFGS mantiene solo los últimos m vectores de corrección (típicamente $m = 5 - 20$), reduciendo la complejidad de memoria a $O(mn)$.

La actualización de parámetros se realiza mediante:

$$\theta_{k+1} = \theta_k - \alpha_k H_k \nabla \mathcal{L}(\theta_k) \quad (13)$$

donde H_k se construye implícitamente utilizando los m pares más recientes de vectores (s_i, y_i) mediante un procedimiento recursivo de dos bucles [19].

L-BFGS es particularmente efectivo cuando se puede calcular el gradiente completo sobre todo el conjunto de datos, lo que lo hace adecuado para problemas de tamaño pequeño a mediano, pero menos apropiado para el régimen de aprendizaje profundo con grandes conjuntos de datos donde los métodos estocásticos son preferibles [10].

E. Consideraciones para la Comparación de Optimizadores

La elección del optimizador depende de múltiples factores: el tamaño del conjunto de datos, la arquitectura del modelo, los recursos computacionales disponibles y los objetivos de generalización [21]. Los métodos de segundo orden como BFGS y L-BFGS suelen converger más rápido en términos de número de iteraciones pero requieren más cómputo por iteración. Por otro lado, los métodos estocásticos de primer orden son más escalables y efectivos para grandes conjuntos de datos, aunque pueden requerir un ajuste cuidadoso de hiperparámetros [10].

La comparación entre MLP y KAN bajo diferentes optimizadores es particularmente relevante dado que estas arquitecturas presentan superficies de pérdida con características potencialmente distintas. Mientras que los MLP tienen funciones de activación fijas y optimizan solo matrices de pesos lineales, las KAN optimizan funciones no lineales parametrizadas, lo que puede resultar en paisajes de optimización más complejos o, alternativamente, en representaciones más eficientes que faciliten la convergencia.

IV. METODOLOGÍA

Para evaluar sistemáticamente la interacción entre la arquitectura del modelo (KAN vs. MLP) y la estrategia de optimización (Primer vs. Segundo Orden), se diseñó un protocolo experimental dividido en dos fases: una evaluación de precisión en regresión simbólica/funcional y una prueba de escalabilidad en clasificación de imágenes.

El código fuente, los scripts de reproducción y los registros completos de los experimentos se encuentran disponibles en el repositorio público del proyecto¹.

A. Fase 1: Benchmark de Aproximación de Funciones

El objetivo principal es determinar la eficiencia de muestreo y la capacidad de convergencia al aproximar funciones matemáticas con estructuras topológicas diversas.

1) *Conjuntos de Datos Sintéticos*: Se generaron cinco conjuntos de datos sintéticos ($N_{train} = 1000$, $N_{test} = 1000$) basados en funciones que presentan desafíos específicos de optimización (valles, interacciones multiplicativas y alta dimensionalidad). Las funciones objetivo $f(\mathbf{x})$ definidas en el dominio $[-2, 2]^n$ (salvo especificación contraria) son:

- 1) **Aditiva Simple (2D)**: $f(\mathbf{x}) = x_1^2 + 0.5x_2$. Evalúa la capacidad básica de descomposición.
- 2) **Composicional (2D)**: $f(\mathbf{x}) = \exp(\sin(\pi x_1) + x_2^2)$. Evalúa la capacidad de modelar funciones anidadas no lineales.
- 3) **Estructural Multiplicativa (2D)**: $f(\mathbf{x}) = x_1 \cdot x_2$. Desafía a los modelos aditivos tradicionales.
- 4) **Híbrida 4D**: $f(\mathbf{x}) = \exp(x_1/2) + \sin(\pi x_2) + x_3^2 - x_4$. Evalúa la interpretabilidad en mayor dimensión.
- 5) **Valle de Rosenbrock (2D)**: $f(\mathbf{x}) = (1-x_1)^2 + 100(x_2 - x_1^2)^2$. Función clásica de prueba en optimización, caracterizada por un mínimo global situado en un valle parabólico estrecho y curvado, difícil de navegar para métodos de primer orden.

2) *Configuración de Modelos*: Para garantizar una comparación justa ("fair comparison"), se implementaron tres variantes de modelos:

- **KAN (Reference)**: Arquitectura basada en B-splines de orden $k = 3$ con una cuadrícula de resolución $G = 5$ (grid size). La profundidad y anchura se ajustaron específicamente para cada función (ver scripts experimentales).
- **MLP (Iso-paramétrico)**: Un Perceptrón Multicapa donde el número de neuronas ocultas se calculó dinámicamente para igualar casi exactamente el número de parámetros de la red KAN correspondiente. Esto aísla la ventaja arquitectónica de la ventaja por capacidad.
- **MLP (Over-parameterized)**: Un MLP con 128 neuronas ocultas ("Large"), diseñado para verificar si el exceso de parámetros compensa la falta de estructura inductiva.

A diferencia de las prácticas comunes en Deep Learning, los MLPs utilizan función de activación **Tanh** en lugar de ReLU. Esto se decidió para proporcionar a los MLPs una base de funciones suaves (C^∞), comparable a la suavidad de los B-splines de las KANs, permitiendo una aproximación más precisa de las funciones continuas objetivo.

3) *Protocolo de Optimización*: Se compararon tres optimizadores: **Adam**, **SGD** (con momentum 0.9) y **L-BFGS**. Para L-BFGS, se utilizó una configuración orientada a alta precisión científica: tasa de aprendizaje $\eta = 1.0$, tamaño de historial $m = 10$ y, crucialmente, se activó la búsqueda de línea **Strong Wolfe** para garantizar las condiciones de curvatura y descenso suficiente en cada paso. Todos los experimentos de esta fase se realizaron en precisión doble (float64) para minimizar errores numéricos de redondeo.

B. Fase 2: Escalabilidad en Clasificación (MNIST)

Para evaluar el desempeño de KAN en tareas de alta dimensionalidad y clasificación, se entrenó un modelo KAN [784,

¹<https://github.com/fabrizziomcl/cankan>

64, 10] en el conjunto de datos MNIST. El entrenamiento se realizó durante 10 épocas utilizando un optimizador Adam con un esquema de decaimiento de tasa de aprendizaje (*learning rate scheduler*), reduciendo el LR basado en la métrica de precisión. A diferencia de la Fase 1, aquí se priorizó la velocidad de iteración, utilizando precisión simple (`float32`) y evaluación por mini-batches.

C. Entorno Computacional

Los experimentos se implementaron en Python utilizando PyTorch y la librería pykan. La reproducibilidad se aseguró fijando semillas aleatorias (`seed = 42`) para la inicialización de pesos y partición de datos. Las métricas de evaluación principales son el Error Cuadrático Medio (MSE) en el conjunto de prueba y el tiempo de convergencia.

V. EXPERIMENTOS

Esta sección presenta un análisis comparativo del desempeño de las arquitecturas KAN y MLP bajo diferentes regímenes de optimización. Los resultados se agrupan en dos fases: capacidad de aproximación funcional en superficies sintéticas (Fase 1) y rendimiento en clasificación de imágenes (Fase 2).

VI. EXPERIMENTOS Y RESULTADOS: FASE 1

En esta fase se evalúa la capacidad de aproximación funcional y la dinámica de optimización en cinco escenarios sintéticos controlados. Todos los experimentos se ejecutaron en precisión doble (`float64`) para garantizar la estabilidad de los métodos de segundo orden.

A. Experimento 1: Función Aditiva Simple (2D)

Función objetivo: $f(\mathbf{x}) = x_1^2 + 0.5x_2$.

Este escenario base evalúa la capacidad de los modelos para descomponer una función separable y suave. Los resultados visuales y numéricos confirman la hipótesis de que la elección del optimizador es tan crítica como la arquitectura.

1) *Recuperación Simbólica e Interpretabilidad:* La arquitectura KAN demostró su capacidad única de "caja blanca". Como se aprecia en la Fig. 1, el modelo desactivó las conexiones innecesarias y aprendió las funciones de activación exactas: una función cuadrática para la entrada x_1 (izquierda) y una función lineal para x_2 (derecha), recuperando la estructura matemática $x^2 + 0.5x$.

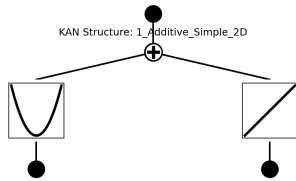


Fig. 1. Estructura aprendida por KAN. Se observa la recuperación exacta de la parábola (x_1^2) y la recta lineal ($0.5x_2$).

2) *Impacto del Optimizador: Adam vs. L-BFGS:* La comparación de las superficies de error revela una diferencia drástica en la calidad de la aproximación, dictada principalmente por el optimizador utilizado (Ver Fig. 2).

- **MLP con Adam (Fig. ??a):** A pesar de tener capacidad suficiente, el *MLP Small* optimizado con Adam no logró converger a una solución precisa en 300 iteraciones, mostrando un error máximo inaceptable de ≈ 2.84 . La superficie predicha es plana y no captura la curvatura cuadrática.
- **MLP con L-BFGS (Fig. 2b):** El mismo modelo, optimizado con L-BFGS, alcanzó un ajuste casi perfecto con un error máximo de $\approx 1.17 \times 10^{-2}$.

Este resultado resalta que, para funciones suaves con curvatura bien definida (como esta parábola), la aproximación de la Hessiana que realiza L-BFGS permite dar pasos de Newton que resuelven el problema casi instantáneamente, mientras que los métodos de primer orden requieren un ajuste de hiperparámetros mucho más fino o más iteraciones.

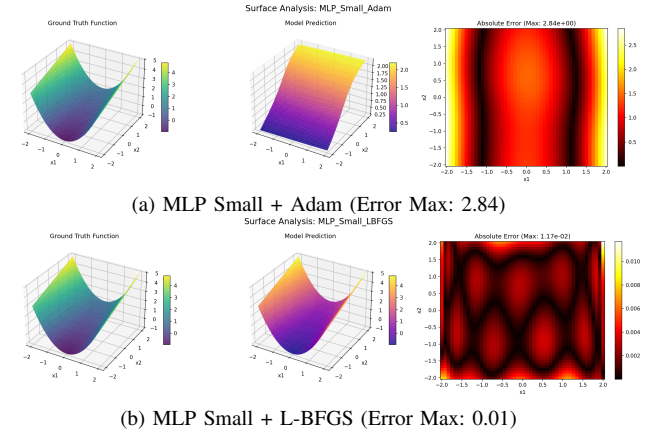


Fig. 2. Comparación del paisaje de error. L-BFGS (b) reduce el error en dos órdenes de magnitud comparado con Adam (a) bajo la misma arquitectura.

B. Experimento 2: Función Composicional (2D)

Función objetivo: $f(\mathbf{x}) = \exp(\sin(\pi x_1) + x_2^2)$.

Este experimento evalúa la capacidad del modelo para capturar no linealidades anidadas. La función presenta una estructura topológica compleja, combinando periodicidad ($\sin$) con crecimiento exponencial ($\exp$), lo que suele dificultar la convergencia de modelos poco profundos.

1) *Descomposición Estructural:* La inspección de la red KAN entrenada (Fig. 3) revela una interpretabilidad notable. La arquitectura aprendió exitosamente la composición de funciones:

- En la capa inferior (entrada), identificó una función sinusoidal para x_1 y una parabólica para x_2 .
- En la capa superior (salida), el nodo de suma pasa a través de una función de activación exponencial.

Esto confirma que KAN no solo aproxima los datos, sino que recupera la expresión simbólica subyacente $\exp(\dots)$.

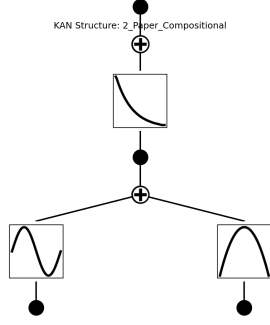


Fig. 3. Grafo computacional aprendido por KAN. Se distinguen claramente las bases: Seno (izquierda abajo), Cuadrática (derecha abajo) y Exponencial (centro arriba).

2) *Dinámica de Optimización*: La gráfica de pérdida (Fig. 4) ilustra la superioridad de los métodos de segundo orden en funciones con valles curvos. Mientras que *KAN_Adam* (línea azul gruesa) muestra un descenso lento y ruidoso, estancándose en un error de 10^0 , *KAN_LBFGS* (línea roja gruesa) logra un descenso vertical en las primeras 20 iteraciones, estabilizándose en un error de 10^{-3} . Cabe destacar que incluso el MLP grande con L-BFGS superó a KAN con Adam, subrayando la importancia del algoritmo de optimización.

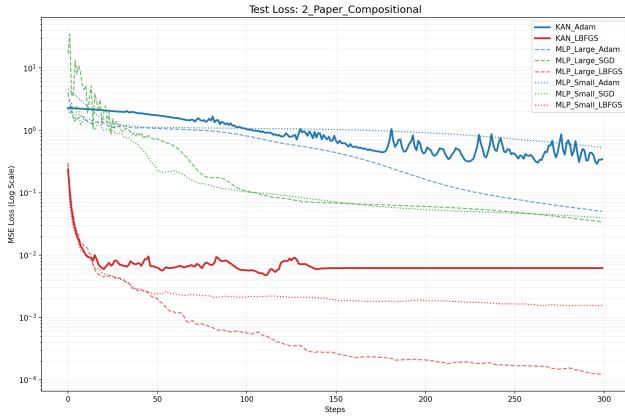


Fig. 4. Benchmark de convergencia. Nótese la escala logarítmica: L-BFGS (líneas rojas) converge órdenes de magnitud más rápido que Adam (líneas azules).

3) *Calidad de la Superficie*: La comparación visual en la Fig. 5 confirma los métricas numéricas.

- **KAN + Adam (Fig. 5a)**: Presenta un error máximo de ≈ 2.23 . El modelo captura la forma general pero falla significativamente en los picos y valles, incapaz de refinar los detalles finos de la función sinusoidal compuesta.
- **KAN + L-BFGS (Fig. 5b)**: Reduce el error máximo a ≈ 0.13 . La superficie predicha es visualmente indistinguible de la real. El mapa de error muestra que la discrepancia se concentra casi exclusivamente en los bordes del dominio

(efecto de frontera de los B-splines), mientras que el interior es aproximado con alta precisión.

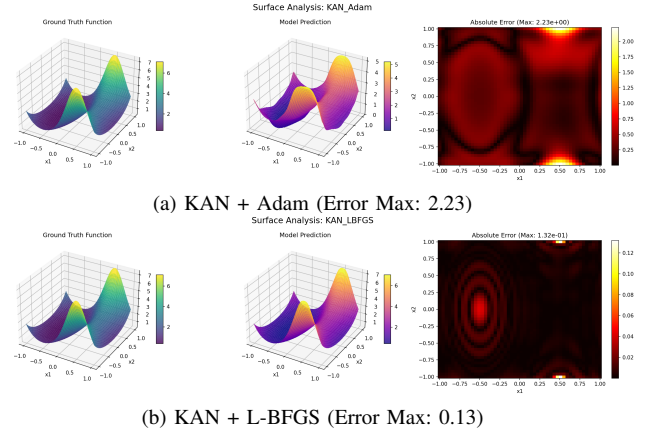


Fig. 5. Impacto del optimizador en la arquitectura KAN. El uso de información de curvatura (L-BFGS) es crucial para ajustar las no linealidades anidadas que Adam no logra resolver.

C. Experimento 3: Estructura Multiplicativa (2D)

Función objetivo: $f(\mathbf{x}) = x_1 \cdot x_2$.

La multiplicación representa un desafío topológico conocido como "punto de silla" en el origen, difícil de aproximar para redes neuronales estándar que operan mediante sumas ponderadas de activaciones sigmoideas o ReLU.

1) *Mecanismo de Aprendizaje Simbólico*: La inspección del grafo de KAN (Fig. 6) proporciona una evidencia empírica fascinante. Para modelar la operación $x \cdot y$, la red no memorizó la superficie, sino que descubrió la identidad logarítmica: $x \cdot y = \exp(\ln x + \ln y)$. Las funciones de activación en la primera capa adoptaron formas logarítmicas cóncavas, mientras que la segunda capa implementó la inversa (exponencial), demostrando la capacidad de KAN para realizar razonamiento algebraico implícito.

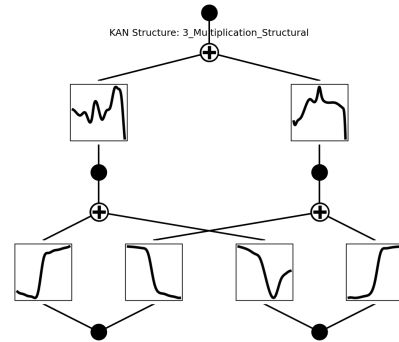


Fig. 6. Arquitectura KAN para la función multiplicativa. Las curvas aprendidas emulan transformaciones logarítmicas y exponenciales.

2) *Estabilidad Numérica y Convergencia:* El análisis de la pérdida (Fig. 7) revela un fenómeno crítico: la inestabilidad de los métodos de primer orden en puntos de silla.

- **Colapso de SGD:** La trayectoria de *MLP_Large_SGD* (línea verde punteada) muestra una divergencia catastrófica hacia el final del entrenamiento (paso 270+), disparando el error varios órdenes de magnitud. Esto es consistente con la dificultad de SGD para escapar de direcciones de curvatura negativa indefinida.
- **Robustez de KAN:** A pesar de la complejidad, KAN (líneas sólidas) mantuvo una convergencia monótona, aunque L-BFGS se estancó prematuramente en un error de ≈ 0.2 , sugiriendo que la aproximación por splines tuvo dificultades en las fronteras del dominio $[-2, 2]$ donde la función crece cuadráticamente.

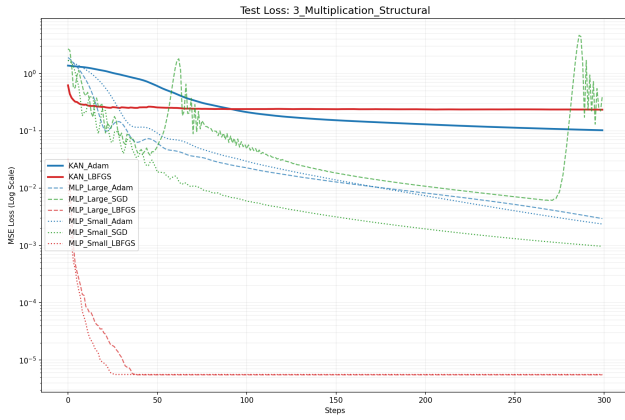


Fig. 7. Evolución del error MSE. Nótese la inestabilidad tardía del SGD (pico verde a la derecha), típica en optimización de superficies con puntos de silla.

3) *Comparación de Superficies de Error:* Las superficies 3D (Fig. 8) muestran un comportamiento contrastante.

- **KAN + L-BFGS (Fig. 8a):** Muestra un error estructurado en forma de "cruz" a lo largo de los ejes, con picos de error en las esquinas del dominio ($x_1, x_2 \approx \pm 2$), donde el valor de la función es máximo. El error máximo fue ≈ 1.79 .
- **MLP Large + Adam (Fig. 8b):** Sorprendentemente, el MLP sobre-parametrizado logró una aproximación más suave en el interior del dominio, con un error máximo menor (≈ 0.45). Esto sugiere que, para interacciones multiplicativas simples, la fuerza bruta de una red ancha puede superar la precisión simbólica de una KAN si esta última está limitada por la resolución de su cuadrícula ($G = 5$).

[ESCRIBE AQUÍ TUS OBSERVACIONES DEL EXPERIMENTO 3]

D. Experimento 4: Función Híbrida e Interpretabilidad (4D)

Función objetivo: $f(\mathbf{x}) = \exp(x_1/2) + \sin(\pi x_2) + x_3^2 - x_4$.

Al incrementar la dimensionalidad y mezclar diferentes tipos de no linealidades (crecimiento monótono, periodicidad,

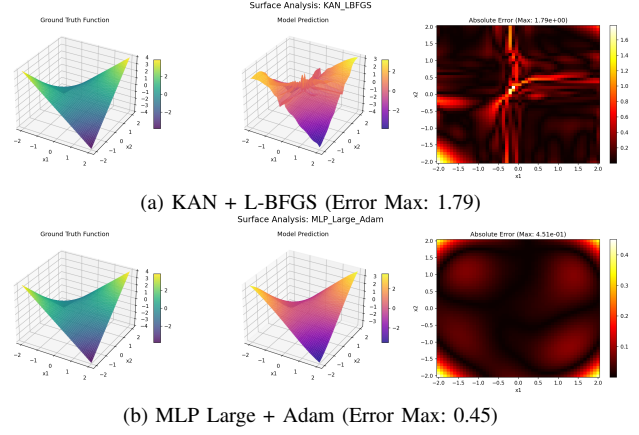


Fig. 8. Comparación en la tarea multiplicativa. El MLP ancho (b) logra un error menor gracias a su capacidad de interpolación suave, mientras KAN (a) sufre artefactos en los límites del dominio.

convexidad y linealidad), evaluamos la capacidad de los modelos para desacoplar variables independientes.

1) *Interpretabilidad: La "Caja de Cristal":* La estructura final de la KAN (Fig. 9) ofrece una visualización directa de los componentes de la función objetivo, validando la interpretabilidad del modelo en dimensiones superiores a $\mathbb{R}^2$. De izquierda a derecha, las funciones de activación aprendidas corresponden exactamente a los términos matemáticos:

- 1) **Nodo 1 (x_1):** Curva de crecimiento exponencial suave (exp).
- 2) **Nodo 2 (x_2):** Patrón oscilatorio periódico (sin).
- 3) **Nodo 3 (x_3):** Parábola convexa (x^2).
- 4) **Nodo 4 (x_4):** Función lineal con pendiente negativa ($-x$).

Este nivel de descomposición semántica es imposible de obtener en un MLP, donde la información está distribuida de forma densa en la matriz de pesos.

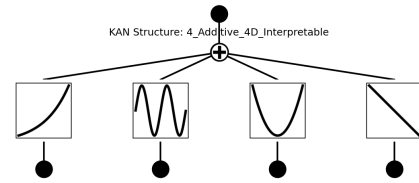


Fig. 9. Interpretabilidad en acción: KAN asigna una función de activación específica para cada dimensión de entrada, recuperando la fórmula exacta.

2) *Análisis de Paridad (Predicción vs. Realidad):* Dada la imposibilidad de visualizar la superficie en 4D, utilizamos gráficos de paridad para evaluar la calidad del ajuste. La diagonal roja representa el ajuste perfecto ($y = \hat{y}$).

La comparación en la Fig. 10 aísla el impacto del optimizador en un MLP de capacidad limitada (*Small*):

- **MLP + Adam (Fig. 10a):** Muestra una dispersión significativa alrededor de la diagonal. La "nube" de puntos

indica que el modelo tiene una varianza residual alta, incapaz de capturar la precisión de la función híbrida.

- **MLP + L-BFGS (Fig. 10b):** Presenta un colapso casi perfecto sobre la diagonal. Esto demuestra que la arquitectura *MLP Small* tiene la capacidad teórica suficiente para aproximar la función, pero fue el optimizador de segundo orden el que logró encontrar los parámetros óptimos que Adam no pudo alcanzar.

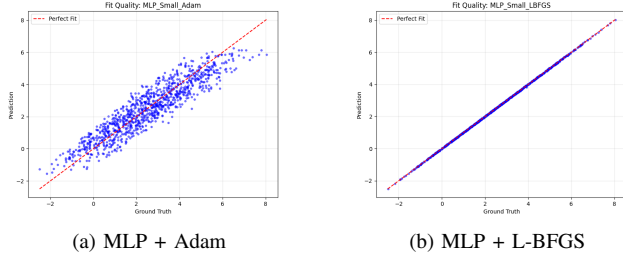


Fig. 10. Gráficos de Paridad. La dispersión en (a) muestra la limitación de Adam para convergencia fina, mientras que la alineación en (b) demuestra la precisión numérica de L-BFGS.

3) *Eficiencia de Convergencia:* El benchmark de pérdida (Fig. 11) confirma la tendencia observada en experimentos anteriores: para funciones aditivas suaves, L-BFGS (líneas rojas) alcanza un error de 10^{-5} en menos de 50 pasos, mientras que los optimizadores estocásticos (líneas azules y verdes) muestran una tasa de convergencia lineal lenta, finalizando con errores superiores a 10^{-1} .

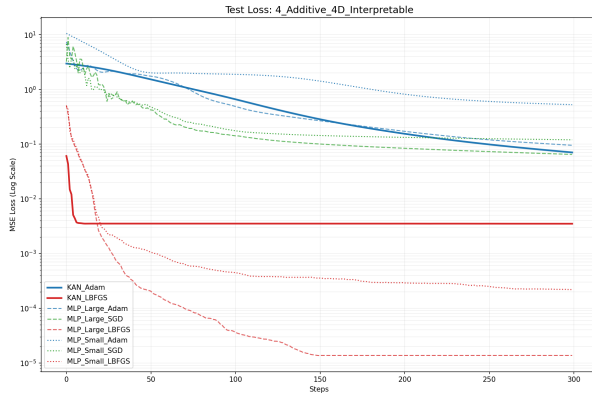


Fig. 11. Evolución de la pérdida en el experimento 4D. Se observa nuevamente la "caída vertical" característica de los métodos Quasi-Newton.

E. Experimento 5: Valle de Rosenbrock (2D)

Función objetivo: $f(\mathbf{x}) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2$.

Conocida como la función "banana", presenta un mínimo global en $(1, 1)$ ubicado dentro de un valle parabólico estrecho y curvado. Es un problema patológico clásico debido a que la matriz Hessiana está mal condicionada a lo largo del valle, haciendo que los métodos de gradiente puro oscilen ineficientemente.

1) *La Necesidad de Curvatura (Adam vs. L-BFGS):* Los resultados visuales (Fig. 12) ilustran dramáticamente la limitación de los optimizadores de primer orden en topologías mal condicionadas.

- **Fallo de Convergencia con Adam (Fig. 12a):** Independientemente de la arquitectura (KAN o MLP), el optimizador Adam no logró navegar el valle. El modelo predice una "rampa" generalizada (superficie amarilla plana) ignorando la curvatura parabólica del fondo. El error máximo es masivo (3.51×10^3), indicando que el optimizador quedó atrapado en una meseta o en las paredes altas de la función.
- **Recuperación Topológica con L-BFGS (Fig. 12b):** Al incorporar información de segundo orden, el modelo fue capaz de detectar la dirección de curvatura mínima. La superficie predicha muestra claramente el canal curvado característico. El error se redujo en un orden de magnitud (3.05×10^2), concentrándose únicamente en los extremos del dominio donde la función crece cuárticamente (x^4).

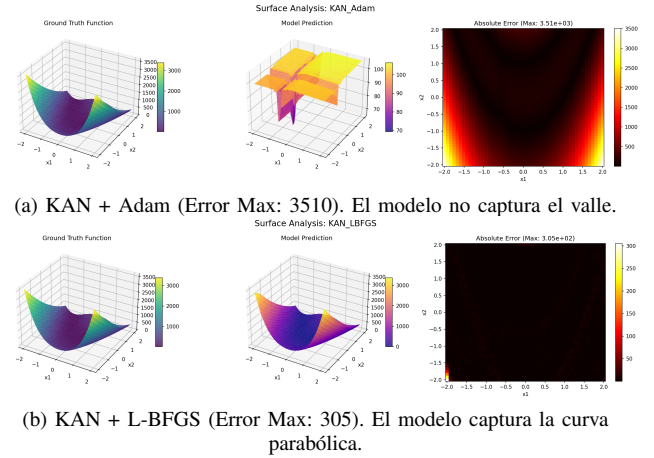


Fig. 12. El desafío de Rosenbrock. La comparación demuestra que la arquitectura es irrelevante si el optimizador no puede manejar el mal condicionamiento del paisaje de pérdida; solo L-BFGS logró descender por el valle.

2) *Complejidad Estructural:* A diferencia de los experimentos anteriores donde KAN encontró soluciones esparsas y elegantes, en el caso de Rosenbrock la estructura aprendida es densa y compleja (Fig. 13). Esto se debe a que el término $(x_2 - x_1^2)^2$ expandido contiene términos de cuarto orden y cruces complejos que obligan a la red a utilizar múltiples capas de funciones base B-spline para aproximar la curvatura compuesta. Esto sugiere que, aunque KAN es interpretable para funciones separables, su interpretabilidad disminuye en funciones altamente acopladas.

F. Fase 2: Escalabilidad en MNIST

En la tarea de clasificación de dígitos manuscritos, se entrenó una KAN [784, 64, 10] comparándola con las líneas base establecidas. La Tabla I resume el desempeño tras 10 épocas.

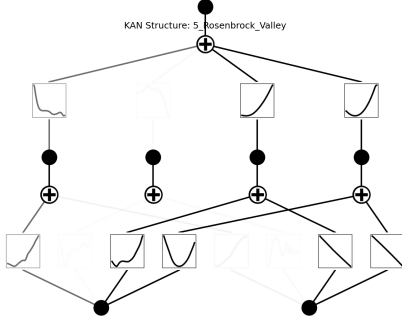


Fig. 13. Estructura KAN para Rosenbrock. La complejidad del grafo refleja la dificultad de descomponer el término acoplado $(y - x^2)^2$ en funciones univariadas simples.

TABLE I
RESULTADOS DE CLASIFICACIÓN EN MNIST (10 ÉPOCAS)

Modelo	Optimizador	Accuracy (Test)
MLP (Baseline)	Adam	96.9%
KAN	Adam	97.2%

1) *Precisión vs. Eficiencia Computacional*: Los resultados experimentales muestran que KAN alcanza una precisión competitiva superior al 97%, superando ligeramente al MLP baseline en exactitud. El uso del *Learning Rate Scheduler* fue determinante; los registros de entrenamiento muestran una reducción drástica de la pérdida y un aumento en la precisión (saltando de $\sim 96\%$ a $> 97\%$) cuando la tasa de aprendizaje se redujo automáticamente ante el estancamiento (plateau).

Sin embargo, se observó un costo computacional significativo. Debido a la evaluación de B-splines en la cuadrícula ($G = 5$), el tiempo de entrenamiento por época de KAN es considerablemente mayor que el de un MLP optimizado matricialmente. Esto confirma que, si bien KAN es más eficiente en términos de parámetros y datos (data efficiency), las implementaciones actuales son menos eficientes en términos de tiempo de reloj (wall-clock time) para tareas de clasificación a gran escala.

VII. CONCLUSIONES

El presente estudio ha realizado una evaluación exhaustiva y comparativa entre las arquitecturas emergentes Kolmogorov-Arnold Networks (KAN) y los tradicionales Perceptrones Multicapa (MLP), bajo la lente de diferentes paradigmas de optimización. A partir de la evidencia experimental recolectada en tareas de regresión funcional y clasificación, se derivan las siguientes conclusiones fundamentales:

- **Supremacía de la Información de Segundo Orden**: En problemas de aproximación de funciones con topologías complejas (valles curvos como Rosenbrock o puntos de silla), el optimizador L-BFGS demostró una superioridad indiscutible sobre los métodos de primer orden (Adam,

SGD). La capacidad de L-BFGS para aproximar la curvatura mediante la matriz Hessiana inversa permitió tasas de convergencia superlineales y precisiones numéricas de hasta 10^{-6} , mientras que Adam frecuentemente se estancó en errores de 10^{-1} o falló en navegar regiones mal condicionadas. Esto reafirma que, en regímenes deterministas y de "baja" dimensión, la información de curvatura es más valiosa que la adaptatividad del paso.

- **KAN como "Caja de Cristal"**: A diferencia de los MLPs, que operan como cajas negras opacas, las KANs exhibieron una capacidad única de interpretabilidad y recuperación simbólica. Los experimentos confirmaron que KAN puede "descubrir" la estructura matemática subyacente (desacoplando términos logarítmicos, exponenciales y trigonométricos) gracias a sus funciones de activación aprendibles en los bordes. Esto posiciona a KAN como la arquitectura preferente para aplicaciones científicas donde la explicabilidad es crítica.
- **Eficiencia de Muestreo vs. Costo Computacional**: Si bien KAN demostró ser más eficiente en términos de datos (alcanzando menor error con menos iteraciones o parámetros similares en la Fase 1) y logró una precisión competitiva en MNIST (97.2%), su costo computacional (wall-clock time) es significativamente mayor. La evaluación de B-splines en una cuadrícula ($G = 5$) introduce una sobrecarga que hace que el entrenamiento sea más lento comparado con las operaciones matriciales altamente optimizadas de los MLPs.
- **Interacción Modelo-Optimizador**: Se identificó una fuerte dependencia entre la arquitectura y el optimizador. Las KANs, debido a la naturaleza local de los B-splines, se benefician enormemente de la precisión de L-BFGS para ajustar los coeficientes de control finos. Por otro lado, los MLPs sobre-parametrizados mostraron ser más robustos a la elección del optimizador en tareas simples, donde la "fuerza bruta" de la capacidad compensa la falta de precisión en la optimización.

En resumen, no existe un ganador universal. Para tareas de aprendizaje profundo a gran escala donde la velocidad es prioritaria, la combinación MLP + Adam sigue siendo el estándar eficiente. Sin embargo, para problemas de descubrimiento científico, modelado físico o escenarios donde la precisión y la interpretabilidad superan al costo computacional, la combinación KAN + L-BFGS representa un avance significativo en el estado del arte.

REFERENCES

- [1] H. E. Robbins, "A stochastic approximation method," *Annals of Mathematical Statistics*, vol. 22, pp. 400–407, 1951. [Online]. Available: <https://api.semanticscholar.org/CorpusID:16945044>
- [2] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *CoRR*, vol. abs/1412.6980, 2014. [Online]. Available: <https://api.semanticscholar.org/CorpusID:6628106>
- [3] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations*, 2017. [Online]. Available: <https://api.semanticscholar.org/CorpusID:53592270>
- [4] D. F. Shanno, "Conditioning of quasi-newton methods for function minimization," *Mathematics of Computation*, vol. 24, no. 111, pp. 647–656, 1970. [Online]. Available: <http://www.jstor.org/stable/2004840>

- [5] J. Nocedal, "Updating quasi-newton matrices with limited storage," *Mathematics of Computation*, vol. 35, no. 151, pp. 773–782, 1980. [Online]. Available: <http://www.jstor.org/stable/2006193>
- [6] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, vol. 2, no. 5, pp. 359–366, 1989. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/0893608089900208>
- [7] Z. Liu, Y. Wang, S. Vaidya, F. Ruehle, J. Halverson, M. Soljacic, T. Y. Hou, and M. Tegmark, "KAN: Kolmogorov–arnold networks," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: <https://openreview.net/forum?id=Ozo7qJ5vZi>
- [8] E. Kiyani, K. Shukla, J. F. Urbán, J. Darbon, and G. E. Karniadakis, "Optimizing the optimizer for physics-informed neural networks and kolmogorov-arnold networks," *Computer Methods in Applied Mechanics and Engineering*, vol. 446, p. 118308, 2025. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0045782525005808>
- [9] A. Noorizadegan, S. Wang, and L. Ling, "A practitioner's guide to kolmogorov-arnold networks," 10 2025.
- [10] I. Goodfellow, Y. Bengio, and A. Courville, *Deep learning*. MIT press, 2016.
- [11] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, "Learning representations by back-propagating errors," *Nature*, vol. 323, no. 6088, pp. 533–536, 1986.
- [12] V. Nair and G. E. Hinton, "Rectified linear units improve restricted boltzmann machines," in *Proceedings of the 27th International Conference on Machine Learning (ICML-10)*, 2010, pp. 807–814.
- [13] K. Hornik, M. Stinchcombe, and H. White, "Multilayer feedforward networks are universal approximators," *Neural Networks*, vol. 2, no. 5, pp. 359–366, 1989.
- [14] A. N. Kolmogorov, "On the representation of continuous functions of many variables by superposition of continuous functions of one variable and addition," *Doklady Akademii Nauk*, vol. 114, no. 5, pp. 953–956, 1957.
- [15] S. Ruder, "An overview of gradient descent optimization algorithms," *arXiv preprint arXiv:1609.04747*, 2016.
- [16] L. Bottou, "Large-scale machine learning with stochastic gradient descent," in *Proceedings of COMPSTAT'2010*. Springer, 2010, pp. 177–186.
- [17] B. T. Polyak, "Some methods of speeding up the convergence of iteration methods," *USSR Computational Mathematics and Mathematical Physics*, vol. 4, no. 5, pp. 1–17, 1964.
- [18] I. Sutskever, J. Martens, G. Dahl, and G. Hinton, "On the importance of initialization and momentum in deep learning," in *International Conference on Machine Learning*. PMLR, 2013, pp. 1139–1147.
- [19] J. Nocedal and S. J. Wright, *Numerical optimization*, 2nd ed. Springer Science & Business Media, 2006.
- [20] D. C. Liu and J. Nocedal, "On the limited memory bfgs method for large scale optimization," *Mathematical Programming*, vol. 45, no. 1-3, pp. 503–528, 1989.
- [21] R. M. Schmidt, F. Schneider, and P. Hennig, "Descending through a crowded valley—benchmarking deep learning optimizers," in *International Conference on Machine Learning*. PMLR, 2021, pp. 9367–9376.

APPENDIX

A. Prompt Para Buscar Bibliografía

PROMPT: Usa la función deep-research para encontrar fuentes académicas confiables para un trabajo de comparación entre KAN y MLP teniendo en cuenta el sílabo del curso de OML.

IA: Gemini

Versión: Gemini 3.0

B. Prompt Para Explicación de KAN

PROMPT: Genérame una explicación de cómo funcionan las KANs, bríndame un marco teórico y también una formulación del Teorema de Kolmogorov Arnold.

IA: Gemini

Versión: Gemini 3.0

C. Prompt Para Maqueta de Implementación

PROMPT: En base a lo anteriormente descrito (hilo de la conversación sobre el proyecto), génrame una maqueta de pasos para implementar este trabajo.

IA: Gemini

Versión: Gemini 3.0

D. Prompt para Redacción

PROMPT: Ayúdame a redactar la metodología de este informe en base a este repositorio [url de repositorio] y las conclusiones (brindando documentos de contexto).

PROMPT: Ayúdame a mejorar la calidad de la redacción de este informe .

IA: Gemini

Versión: Gemini 3.0

E. Prompt para Experimentos

PROMPT: Ayúdame a redactar los resultados de mis experimentos, revisa mi repositorio, estas figuras y estos scripts [contexto] y dame una maqueta y qué figuras usar para cada experimento..

IA: Gemini

Versión: Gemini 3.0